M10 # Skills

Méthodologie Intermediate

Modules de compétences réutilisables avec ressources embarquées

Structure d'un skill

Un skill est un dossier dans `.claude/skills/` contenant un fichier `SKILL.md` et optionnellement des ressources supplémentaires (docs, templates, scripts, checklists).

```
mon-skill/
├── SKILL.md          # Instructions + frontmatter (obligatoire)
├── reference.md      # Documentation de référence
├── checklists/
│   └── quality.md
└── examples/
    └── pattern.ts
```

Modes d'invocation

Mode	Frontmatter	Comportement
User-invocable	`disable-model-invocation: true`	Déclenché par l'utilisateur via `/nom`
Modèle uniquement	`user-invocable: false`	Masqué dans le menu utilisateur ; Claude peut l'invoquer
Les deux	(aucun flag)	Invocable des deux façons

Frontmatter SKILL.md

```yaml
---
name: security-guardian
description: Expertise OWASP, auth, et protection des données
allowed-tools: Read Grep Bash
---
```

Champ	Rôle
`name`	Kebab-case, 1-64 chars, doit correspondre au dossier
`description`	Déclencheur d'activation ; avec `when_to_use`, tronqué à 1 536 car. dans la liste (1 024 : limite agentskills.io)
`allowed-tools`	Whitelist d'outils (supporte les wildcards)
`argument-hint`	Placeholder dans le menu si le skill accepte `$ARGUMENTS`
`disable-model-invocation`	`true` pour invocation manuelle uniquement
`user-invocable`	`false` masque le skill du menu `/`
`model` / `effort`	Modèle (`inherit` possible) et effort (`low` à `max`) du skill
`context` / `paths`	`fork` : sous-agent ; `paths` : globs limitant l'activation

Skills vs commands

Aspect	Skill	Command
Ressources	Peut embarquer des fichiers	Markdown seul
Portée	Connaissance domaine	Workflow procédural
Réutilisation	Entre agents	Invocation directe
Standard	agentskills.io (portable)	Claude Code uniquement

La différence principale : un skill peut embarquer un fichier `reference.md` de 500 lignes qui devient disponible lors de l'invocation. Une command ne peut pas.

Installer des skills communautaires

```
# Inspecter les skills disponibles avant l'installation
npx skills add vercel-labs/agent-skills --list

# Choisir les skills souhaités en mode interactif
npx skills add vercel-labs/agent-skills --agent claude-code

# Vérifier les skills installés
npx skills list
```

Le CLI skills installe des paquets communautaires. Un catalogue ou un scan ne garantit pas leur sécurité : inspecter les instructions, les scripts, les accès demandés et la révision choisie avant utilisation.

Invocation

Les skills se déclenchent comme les commandes, par leur nom :

```
/security-guardian        # Invocation directe
/tdd                      # Skill TDD
/design-patterns detect   # Avec argument
```

Les agents peuvent précharger des skills via le champ `skills:` dans leur frontmatter. Le contenu est injecté dans le contexte de l'agent au démarrage.

Lifecycle et amélioration

Deux patterns de gestion dans le temps :

Claudeception : méta-skill qui génère de nouveaux skills à partir des découvertes de session. Install :

`git clone https://github.com/blader/Claudeception ~/.claude/skills/claudeception`

Claude Reflect System : améliore les skills existants en analysant les corrections détectées pendant les sessions. Propose les changements avant de les appliquer.

Précaution importante

Claude peut invoquer automatiquement les skills éligibles, mais leur découverte ne garantit pas leur invocation à chaque demande pertinente. Placez les instructions permanentes dans `CLAUDE.md` ou `.claude/rules/` et testez l'activation sur des prompts représentatifs.